

Lecture 01

Introduction & Welcome to 213

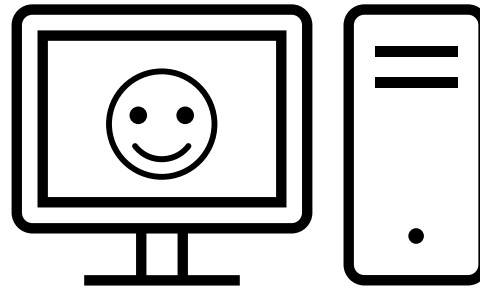
Comp_Sci 213
Introduction to Computer Systems
Kate Smith – Fall 2026

Slides adapted from:

Ghena, St-Amour, Hardavellas, Bustamente (Northwestern), Bryant, O'Hallaron (CMU), Garcia, Weaver (UC Berkeley)

Welcome to Comp_Sci 213!

- **Motivating question:** how does a computer work?
- This class will discuss fundamental concepts that underly all computer systems, showing concrete ways that these ideas affect application program correctness, performance, and utility
- We will explore that question across four major sections:
 - **Representations** of information on a computer
 - How the **machine** executes software
 - How **memory** is organized
 - How the **operating system** manages this all for efficiency and security



Today's Goals

- Introduce the theme and goals of the course
- Describe course logistics

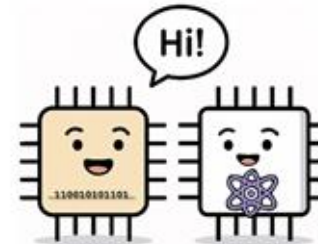
Kate Smith (she/her)

- Assistant Professor, Computer Science
- Education (Southern Methodist University)
 - BS, Mathematics and Electrical Engineering
 - MSEE
 - PhD, Electrical Engineering
- Interests:
 - Quantum Computing, Systems, Software, and Architecture
- Experience:
 - Postdoc @ UChicago CS
 - Q. Software Manager @ Infleqtion
- Teaching:
 - Foundations of Quantum Computing and Quantum Information
 - Quantum Computer Systems
 - Quantum Applications and Programming
 - Intro to Computer Systems
- Research.....

Quantum Architecture (QuantA) @ NU

Problem: How do we create computing systems that make *quantum advantage* feasible?

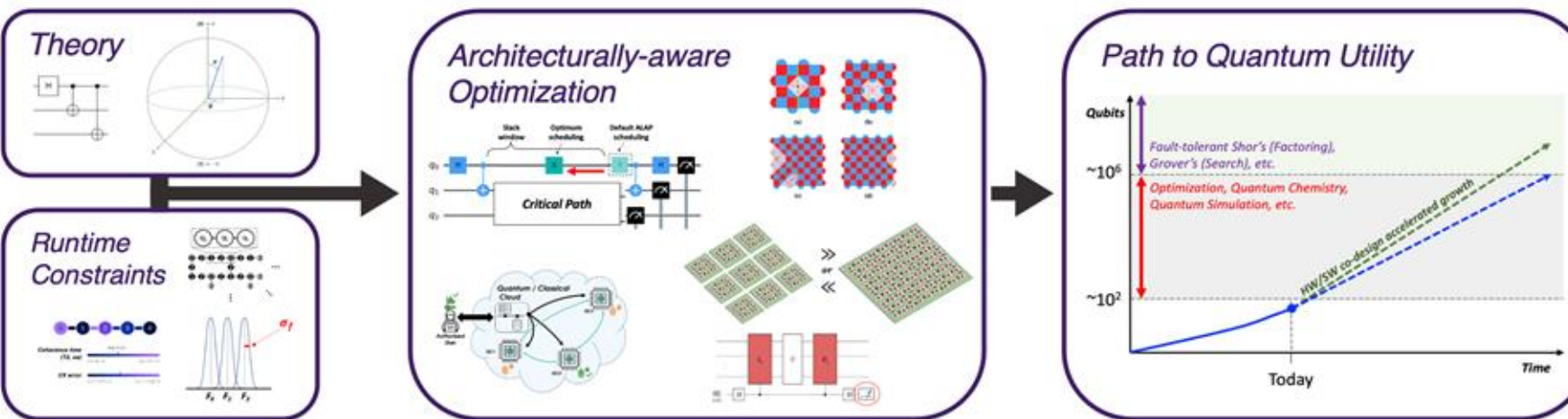
Solution: **QuantA** proposes software frameworks and architecture that make quantum systems more *reliable* and *scalable* through context-aware quantum error management.



Kate Smith

kns@northwestern.edu

Group: quantum-architecture.com
Personal: quantumkate.com



¹A Quantum Computational Compiler and Design Tool for Technology-specific Targets. ISCA 2019.

²Superstaq: Deep Optimization of Quantum Programs. QCE 2023.

³TimeStitch: Exploiting Slack to Mitigate Decoherence in Quantum Circuits. TQC 2022.

⁴VAQEM: A Variational Approach to Quantum Error Mitigation. HPCA 2022.

⁵Decomposition Strategies for Bosonic Qudit Compilation. QCE 2026.

⁶VarSEQ: Variable Strength Error-corrected QRAM. QCE 2026.

⁷Access Improvements to Densely Packed Quantum Memory. QCE 2025 (poster paper).

⁸Scaling Superconducting Quantum Computers with Chiplet Architectures. MICRO 2022.

⁹Codisign of Quantum Error-correcting Codes and Modular Chiplets in the Presence of Defects. ASPLOS 2024.

¹⁰SEQC: Stratify-Elaborate Quantum Compilation Towards Modular Hybrid Architectures. QCE 2026.

¹¹Dynamic Resource Allocation with Quantum Error Detection. arXiv:2408.05565.

¹²Extrapolating Pauli Checks for Expectation Value Estimation on Noisy Quantum Devices. TQE 2026.

¹³Generalizing Pauli Checks for Qudit-based Quantum Error Detection and Mitigation. QCE 2026.

Architecture-aware Compilation

- Synthesis verification with QMDDs¹
- Physics-aware deep Superstaq optimization targeting 5 qubit platforms²
- Optimized gate scheduling
 - TimeStitch: 1.6× mean fidelity improvement (1.2× over Dynamical Decoupling alone)³
 - VAQEM: increase in quality of measured VQE objective⁴
- Compilation for qudit-based quantum systems⁵

Optimized Quantum Error Correction

- Heterogeneous QEC encoding and BB-inspired architecture achieves QRAM designs with constant query infidelity scaling while reducing physical qubit overhead⁶
- By considering gliding operations with walking surface code, dense quantum memory can achieve $O(lw)$ better resource efficiency - possible LER improvements are under investigation⁷

Scaling Quantum Systems via Modularity

- Proposed chiplet architectures with yield improvements of $\sim 90\times$ and infidelity reductions of $\sim 0.8\times$ ⁸
- Deformed surface codes on chiplets improve logical error rates (modest) and resource overheads ($45\times$ - $10^5\times$) in the presence of defective qubits and links⁹
- Hierarchical compilation (Stratify/Elaborate) strategies lead to fewer cross-chiplet gates, higher fidelity, and faster compilation¹⁰

Error Detection and Mitigation

- Pauli checks learn noise landscapes during runtime and can implement error mitigation to improve quantum program performance $>1.11\times$ ¹¹
- Pauli check extrapolation combines zero noise extrapolation with Pauli checks - we find “maximum check limit”¹²
- Novel qudit Pauli check sandwiching techniques¹³

Communication

- For this section, course materials can be found on Kate's website
 - <https://knsmith.github.io/nucs-213-f26.html>
- Please ask questions during lecture 😊
 - If a question is extremely in-depth, we can always take discussions offline
- Use Piazza! All questions should go here rather than being sent via email (email response may be slow...)
 - <https://piazza.com/class/mtnchvvpcei7fy>
 - Link can also be found on Canvas
- We will also have a class Slack, which is helpful for collaboration (e.g. for labs)
 - <https://nucs213.slack.com> (there is a link to join on on Canvas)
- Order for questions (outside of lecture time): Piazza/Slack, PMs, TA, Prof

- **Course Themes**
- Logistics

Convenient computing

- Computers operate on integers, reals, structs, arrays, etc.
- Computers operate on variables and functions
- Computers execute conditionals, loops, etc.
- Memory is an infinite bag of objects my program can allocate
- Memory doesn't have to be shared with any other program
- Memory is always equivalently fast to access
- Etc.

Convenient **illusions** in computing

- Computers operate on integers, reals, structs, arrays, etc.
- Computers operate on variables and functions
- Computers execute conditionals, loops, etc.
- Memory is an infinite bag of objects my program can allocate
- Memory doesn't have to be shared with any other program
- Memory is always equivalently fast to access
- Etc.
- None of these are actually true!
 - But we usually program as if they were, and we get away with it!
 - What's going on under the hood?

The power of abstraction

- These illusions are called ***abstractions***
- They approximate reality, but leave out details
 - Instead, they provide an *interface* that we can work and think with
- We can forget about those details, and be more productive
 - Avoid getting "lost in the sauce"
- Why is programming in Python more fun than programming in C?
 - Python abstracts away a lot of details you have to do manually in C
 - Example: memory allocation

The limits of abstraction

- Sometimes, abstractions break down
 - Their implementation is buggy
 - Their performance is inadequate
 - We need control over the details they hide
 - Security concerns make the details important
- At that point, details come rushing back
 - Can't pretend they don't exist anymore
 - We must know how to deal with them
- This class prepares you to be ready when that happens

When do abstractions break?

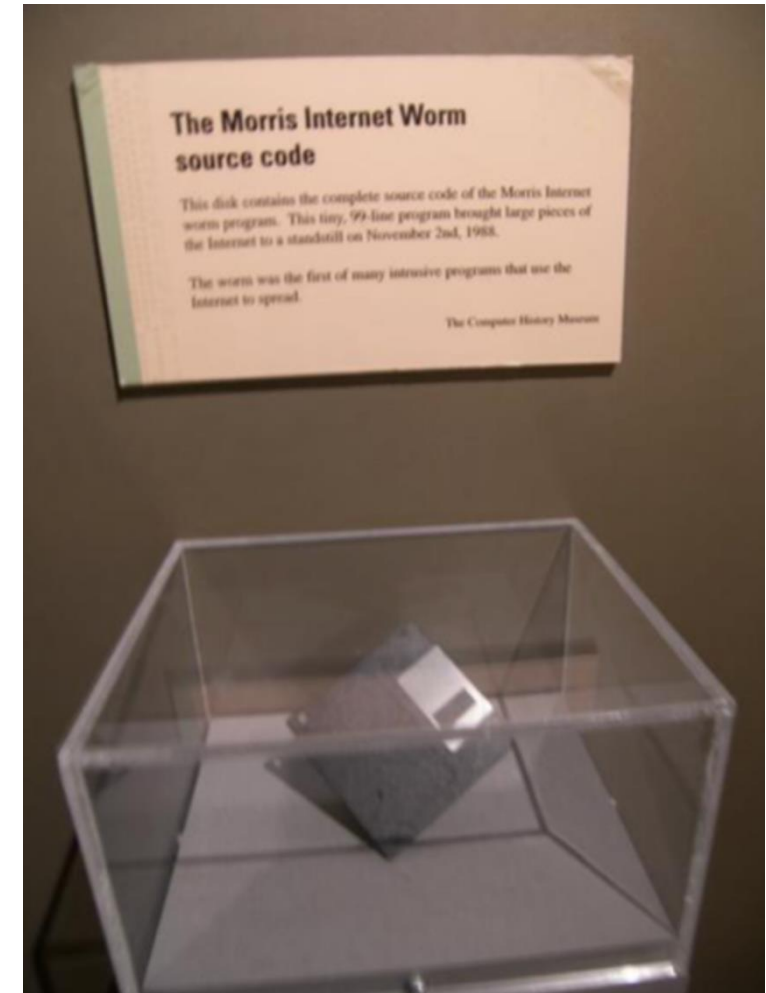
- Let's talk about some real-world examples of “broken” software
 - That broke because of how the underlying system actually works
1. Dates and Times
 2. Network Security
 3. Type conversion
 4. Data leakage
 5. Ordering of Loop Iteration

Complicated designs fail in unexpected ways

- Some software engineers at Microsoft came up with a cute way of storing dates
 - Two-digit year, month, date, hour, minute concatenated into a 10-digit number
 - Example: 2005230710 -> May 23, 2020 at 7:10 AM
- Stored as a C `int` (32-bit signed number)
 - Convenient, because these are easy to use in C/C++
 - Maximum value for an `int`: 2147483647
- Result: Starting January 1st, 2022, Microsoft Exchange email servers could no longer send email
 - 2201010001 is greater than the largest 32-bit number
 - Microsoft had to issue an emergency patch

The Morris worm (1988)

- One of the first computer worms distributed across the internet was created by a Cornell graduate student, Robert Morris
 - **Viruses** require host file (i.e. virus is dormant until file is run) / human interaction
 - **Worms** self-replicate and don't need a host to travel
- Four exploits allowed the worm to affect ~6k of the 60k internet-connected computers at the time within 24 hours (!!)
 - Send mail debug -> bug that allowed remote command execution
 - Fingerd buffer overflow -> allowed remote code execution
 - rsh (remote shell) / rexec (remote execution) with implicit trust -> passwordless access to other machines
 - Weak unix passwords -> obtain access as a legitimate user with relatively few guesses
- The worm was programmed to continue copying itself, even if a machine was already infected, eventually causing denial-of-service attack
 - Files weren't damaged or destroyed



Expectation mismatches lead to real-world problems

- Ariane 5 explosion (1996) ~40 seconds after launch
 - Inertial reference system converted a 64-bit float to a 16-bit integer
 - Expectation: converting from decimal to whole numbers is safe
 - Had worked in the past in Ariane 4, but Ariane 5 was faster
 - Speed too large to fit in a 16-bit integer -> software fault
 - Reality: rocket explodes, and \$370 million is lost

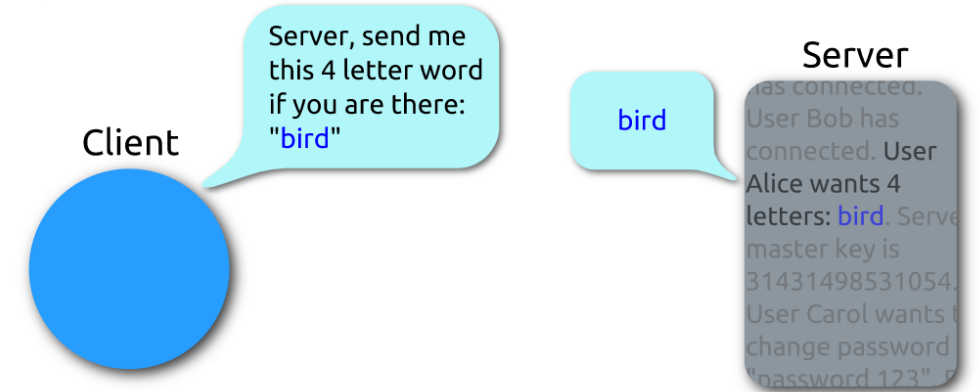


Simple bugs can result in massive vulnerabilities

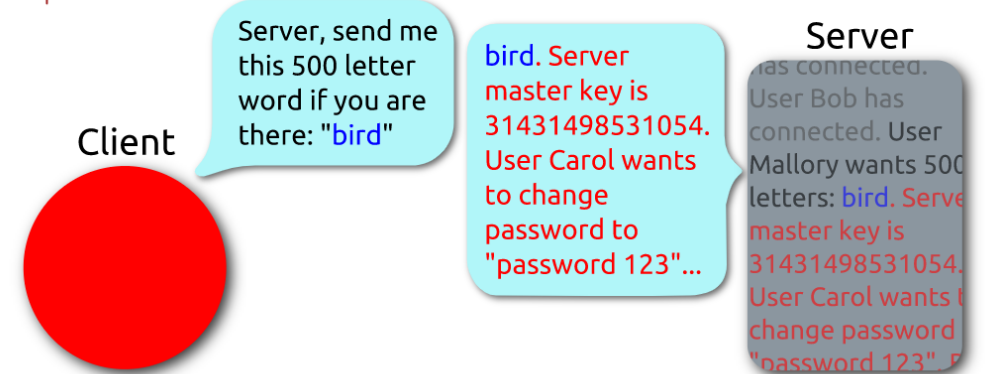
- Heartbleed: 2014 vulnerability in OpenSSL found by Google security
- Normally, clients can check if a server is active by sending a message and listening for echoed response
 - This is referred to as a **TLS heartbeat**
- C library forgot to check bounds of array and could be abused to return important memory
- 17% of servers on the Internet were vulnerable (~500,000 servers)
 - "...worst vulnerability found (at least in terms of its potential impact) since commercial traffic began to flow on the Internet." – Forbes, Joseph Steinberg



Heartbeat – Normal usage



Heartbeat – Malicious usage



Hardware realities impact software performance

- Abstracted lower-level details can affect performance a lot!
- Question: does the order of iterating through an array matter?
 - Each column in a row OR each row in a column?
- Answer: right code is 10-32 times slower on Intel systems
 - Cache design and performance influence this
 - Elements within a row are contiguous in memory!

```
void copyij(int src[4096][4096],
            int dst[4096][4096])
{
    for (int i=0; i<4096; i++){
        for (int j=0; j<4096; j++){
            dst[i][j] = src[i][j];
        }
    }
}
```



```
void copyji(int src[4096][4096],
            int dst[4096][4096])
{
    for (int j=0; j<4096; j++){
        for (int i=0; i<4096; i++){
            dst[i][j] = src[i][j];
        }
    }
}
```

CS213 goals

1. Break through abstractions to understand how computer processors and memories affect software design and performance
2. Introduce concepts of “computer systems” areas:
 - Architecture, Operating Systems, Security, Networks, Compilers, etc.

Course approach

- Most systems courses are builder-centric
 - **Computer Architecture**: design a pipelined processor in Verilog
 - **Operating Systems**: implement portions of an operating system
 - **Compilers**: write a compiler for a simple language
 - **Networking**: implement and simulate network protocols
 - Fun, for sure
 - But ultimately, many more of you will ***build on*** systems
 - Rather than ***build systems*** directly
- This course is programmer-centric
 - Purpose is to show that by knowing more about the underlying system, one can be more effective as a programmer
 - Not just a course for dedicated hackers
 - **We want to bring out the hacker in everyone!**

- Course Themes
- **Logistics**

TOUR OF COURSE References

- Canvas – for turning assignments in and pointers to things
 - *Ignore the quizzes... we won't do these in our section*
- Piazza – for discussion
- Slack – for collaboration and discussion
- Website – for lecture notes, assignments, general information
- Syllabus – for schedule and important course details

Canvas walkthrough

Piazza walkthrough

Slack walkthrough

Website walkthrough

Syllabus walkthrough

Programming Labs

- Four labs
 1. Pack Lab – manipulate bits and bytes of a file
 2. Bomb Lab – deconstruct software to understand it
 3. Attack Lab – exploit security vulnerabilities in software
 4. SETI Lab – make software faster with concurrency
- Work on these as a group of two
 - Work together and don't split up assignments (otherwise you won't learn)
 - Individual is acceptable but less good
 - We'll do a pairing survey if you don't already have a partner in mind
- Very different from CS211 style projects
 - Emphasis on the thinking rather than the programming
- **Lab late policy:** each calendar day after the due date for a lab, 10% is lost. After 1 day, the maximum score is 90%, after 2 days, 80%, etc., for a maximum of 10 days

Lab difficult ranking (ranked by past PMs)

Lab	Difficulty (out of 10)	What is challenging about it?
1. Pack Lab	6	C programming
2. Bomb Lab	9	Interpreting assembly code
3. Attack Lab	5	Debugging what's going wrong
4. SETI Lab	8	C programming AND big codebase

- Be warned: Bomb Lab and SETI Lab are during the busiest parts of the quarter
 - Midterm exam season and last week of quarter, respectively

Homeworks

- Worksheet-style practice problems to help you practice for exams and understand what's going on
 - This class can feel a little like a math class sometimes
 - (But not all the time! I promise)
- Three homeworks that cover class topics
- **No late submissions for homeworks**
 - I want to release their solutions immediately for studying purposes

Exams

- **Midterm** will be in class (see schedule in Syllabus)
- **Final** will be during the exam period (Monday, December 7, 2026 from 9 AM – 11 AM)
- Not cumulative! Exams will cover their own section in the course
- Exams are serious in CS213 - they're how we judge your individual understanding
- ***You need to have a passing grade (64% or higher) averaged between the two exams to pass the course, regardless of your performance in the homework and lab assignments***

AI Usage

- Generative AI tools **MAY NOT** be used to develop **ANY** code
 - They may not be used in any way on lab assignments
- Goal of this course is to develop an intuition for computer systems
- You may use AI tools for studying and understanding course material outside of labs and homeworks

Expectations

- Attend class
- Do your own work
- Learn things 😊